PPL task 7

1. Write a class Car with: Attributes: brand, model A method display_info() that prints brand and model.
Code:

```
class Car:
    def __init__(self, brand, model):
        self.brand = brand
        self.model = model

    def display_info(self):
        print(f"Brand: {self.brand}")
        print(f"Model: {self.model}")

# Example usage
my_car = Car("Toyota", "Corolla")
my_car.display_info()
```

Output:
Brand: Toyota
Model: Corolla

2. Create a class Circle with: Attribute: radius A method area() that returns the area of the circle.
Code:

```
import math

class Circle:
    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return math.pi * self.radius ** 2

# Example usage
c = Circle(5)
print("Area of the circle:", c.area())
```

Output:
Area of the circle: 78.53981633974483

3. Write a class BankAccount with: Attribute: balance (default 0) Methods: o deposit(amount) → increases balance o withdraw(amount) → decreases balance if enough funds are available o get_balance() → returns the current balance

Code:

```
class BankAccount:
    def __init__(self):
        self.balance = 0  # Default balance is 0

    def deposit(self, amount):
        if amount > 0:
            self.balance += amount
            print(f"Deposited: {amount}")
        else:
            print("Deposit amount must be positive.")

    def withdraw(self, amount):
        if amount <= self.balance:
            self.balance -= amount
            print(f"Withdrawn: {amount}")
        else:
            print("Insufficient balance!")

    def get_balance(self):
        return self.balance

# Example usage
account = BankAccount()
account.deposit(1000)
account.withdraw(300)
print("Current balance:", account.get_balance())
```

Output:

```
Deposited: 1000
Withdrawn: 300
Current balance: 700
```

4. Define a class Student with: Attributes: name, marks (list of integers) A method average() that returns the average marks.

Code:

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks  # List of integers

    def average(self):
        if self.marks:
            return sum(self.marks) / len(self.marks)
        else:
            return 0
```

```
# Example usage
student1 = Student("Alice", [85, 90, 78, 92])
print(f"Average marks for {student1.name}:", student1.average())
```

Output:
Average marks for Alice: 86.25

5. Calculator • Methods: • add(a, b) → returns sum • subtract(a, b) → returns difference • multiply(a, b) → returns product • divide(a, b) → returns quotient if b ≠ 0
Code:

```
class Calculator:
    def add(self, a, b):
        return a + b

    def subtract(self, a, b):
        return a - b

    def multiply(self, a, b):
        return a * b

    def divide(self, a, b):
        if b != 0:
            return a / b
        else:
            return "Error: Division by zero"

# Example usage
calc = Calculator()

print("Add: ", calc.add(10, 5))
print("Subtract: ", calc.subtract(10, 5))
print("Multiply: ", calc.multiply(10, 5))
print("Divide: ", calc.divide(10, 0))
```

Output:
Add: 15
Subtract: 5
Multiply: 50
Divide: Error: Division by zero

6. Employee • Attributes: name, salary • Methods: • get_details() → prints employee's name and salary • apply_raise(percent) → increases salary by given percentage
Code:

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary

    def get_details(self):
        print(f"Name: {self.name}")
        print(f"Salary: {self.salary}")

    def apply_raise(self, percent):
        increase = self.salary * (percent / 100)
        self.salary += increase
        print(f"Salary increased by {percent}%. New salary: {self.salary}")

# Example usage
emp = Employee("John Doe", 50000)
emp.get_details()
emp.apply_raise(10)
```

Output:
Name: John Doe
Salary: 50000
Salary increased by 10%. New salary: 55000.0

7. Library • Attributes: list of books (as a list of strings) • Methods: • add_book(title) → adds a book • remove_book(title) → removes a book if it exists • display_books() → prints all available books
Code:

```
class Library:
    def __init__(self):
        self.books = []  # List of book titles

    def add_book(self, title):
        self.books.append(title)
        print(f"Book '{title}' added to the library.")

    def remove_book(self, title):
        if title in self.books:
            self.books.remove(title)
            print(f"Book '{title}' removed from the library.")
        else:
            print(f"Book '{title}' not found in the library.")

    def display_books(self):
        if not self.books:
            print("The library is empty.")
        else:
```

```
        print("Available books in the library:")
        for book in self.books:
            print(f"- {book}")

# Example usage
lib = Library()
lib.add_book("1984")
lib.add_book("To Kill a Mockingbird")
lib.display_books()
lib.remove_book("1984")
lib.display_books()
```

Output:

```
Book '1984' added to the library.
Book 'To Kill a Mockingbird' added to the library.
Available books in the library:
- 1984
- To Kill a Mockingbird
Book '1984' removed from the library.
Available books in the library:
- To Kill a Mockingbird
```